18
Building Responsive Layouts with Flexbox

In this chapter, you'll get a feel for what it's like to lay components out on the screen of mobile devices. Thankfully, React Native polyfills many CSS properties that you might have used in the past to implement page layouts in web applications.

Before you dive into implementing layouts, you'll get a brief introduction to Flexbox and using CSS style properties in React Native apps: it's not quite what you're used to with regular CSS style sheets. Then, you'll implement several React Native layouts using Flexbox.

We will cover the following topics in this chapter:

- Introducing Flexbox
- Introducing React Native styles
- Using the Styled Components library
- Building Flexbox layouts

Technical requirements

You can find the code files present in this chapter on GitHub at https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter18.

Introducing Flexbox

Before the flexible box layout model was introduced to CSS, the various approaches used to build layouts were convoluted and prone to errors. For example, we used **floats**, which were originally intended for text wrapping around images, for table-based layouts. **Flexbox** solves this by abstracting many of the properties that you would normally have to provide in order to make the layout work.

In essence, the Flexbox model is what it probably sounds like to you: a box model that's flexible. That's the beauty of Flexbox: its simplicity. You have a box that acts as a container, and you have **child** elements within that box. Both the container and the **child** elements are flexible in how they're rendered on the screen, as illustrated here:

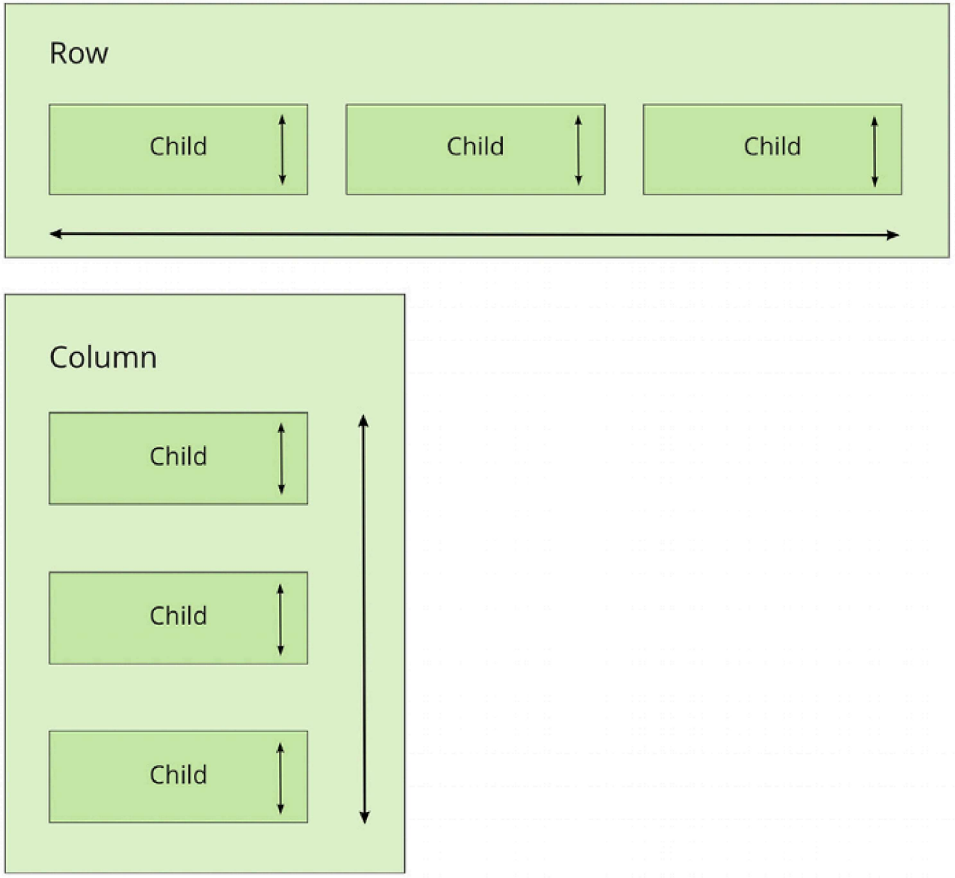

Figure 18.1: Flexbox elements

Flexbox containers have a direction, either column (up/down) or row (left/right). This actually confused me when I was first learning about Flexbox; my brain refused to believe that rows were organized beside each other from left to right. Rows are stacked on top of one another! The key thing to remember is that it's the direction in which the box flexes, not the direction in which boxes are placed on the screen.

For a more in-depth treatment of Flexbox concepts, refer to https://css-tricks.com/snippets/css/a-guide-to-Flexbox.

Now that we've covered the basics of Flexbox layouts at a high level, it's time to learn how styles in React Native applications work.

Introducing React Native styles

It's time to implement your first React Native app, beyond the boilerplate that's generated by **Expo**. I want to make sure that you feel comfortable using React Native style sheets before you start implementing Flexbox layouts in the next section.

Here's what a React Native style sheet looks like:

```
import { Platform, StyleSheet, StatusBar } from "react-native";
export default StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "ghostwhite",
    ...Platform.select({
      ios: { paddingTop: 20 },
      android: { paddingTop: StatusBar.currentHeight },
```

```
      }),
    },
    box: {
      width: 100,
      height: 100,
      justifyContent: "center",
      alignItems: "center",
      backgroundColor: "lightgray",
    },
    boxText: {
      color: "darkslategray",
      fontWeight: "bold",
    },
  });
```

This is a **JavaScript** module, not a CSS module. If you want to declare React Native styles, you need to use plain objects. Then, you call `StyleSheet.create()` and export this from the style module. Note that style names are pretty similar to the web CSS, except that they are written in camel case; for example, `justifyContent` rather than `justify-content`.

As you can see, this style sheet has three styles: `container`, `box`, and `boxText`. Within the `container` style, there's a call to `Platform.select()`:

```
  ...Platform.select({
  ios: { paddingTop: 20 },
```

```
    android: { paddingTop: StatusBar.currentHeight }
  })
```

This function will return different styles based on the platform of the mobile device. Here, you're handling the top padding of the top-level `container` view. You'll probably use this code in most of your apps to make sure that your React components don't render underneath the status bar of the device. Depending on the platform, the padding will require different values. If it's iOS, `paddingTop` is `20`. If it's Android, `paddingTop` will be the value of `StatusBar.currentHeight`.

The preceding `Platform.select()` code is an example of a case where you need to implement a workaround for differences in the platform. For example, if `StatusBar.currentHeight` was available on iOS and Android, you wouldn't need to call `Platform.select()`.

Let's see how these styles are imported and applied to React Native components:

```
import React from "react";
import { Text, View } from "react-native";
import styles from "./styles";
export default function App() {
```

```
    return (
      <View style={styles.container}>
        <View style={styles.box}>
          <Text style={styles.boxText}>I'm in a box</Text>
        </View>
      </View>
    );
  }
```

The styles are assigned to each component via the `style` property. You're trying to render a box with some text in the middle of the screen. Let's make sure that this looks as we expect it to.


13:33
I'm in a box

Figure 18.2: Box in the middle of a screen

We have found out how to apply styles to components using a built-in module, but there is more than one way to define styles. We also have the option to write CSS in React Native. Let's quickly go through it.

Using the Styled Components library

Styled Components is a CSS-in-JS library that styles React Native components using plain CSS. With this approach, you don't need to define style classes via objects and provide style props. The CSS itself is determined via tagged template literals provided by `styled-components`.

To install `styled-components`, run this command in your project:

```
npm install --save styled-components
```

Let's try to rewrite components from the *Introducing React Native styles* section. This is what our `Box` component looks like:

```
import styled from "styled-components/native";
const Box = styled.View'
  width: 100px;
  height: 100px;
  justify-content: center;
  align-items: center;
  background-color: lightgray;
';
const BoxText = styled.Text'
  color: darkslategray;
  font-weight: bold;
';
```

In this example, we've got two components, `Box` and `BoxText`. Now we can use them as usual, but without any other additional styling props:

```
const App = () => {
  return (
    <Box>
      <BoxText>I'm in a box</BoxText>
    </Box>
```

```
    );
  };
```

In further sections, I will use `StyleSheet` objects, but I will avoid `styled-components` for performance reasons. If you want to learn more about `styled-components`, you can read more here: **https://styled-components.com/**.

Perfect! Now that you have an idea of how to set styles on React Native elements, let's use Flexbox to start creating some screen layouts.

Building Flexbox layouts

In this section, you'll learn about several potential layouts that you can use in your React Native applications. I want to stay away from the idea that one layout is better than another. Instead, I'll show you how powerful the Flexbox layout model is for mobile screens so that you can design the kind of layout that best suits your application.

Simple three-column layout

To start things off, let's implement a simple layout with three sections that flex in the column direction (top to bottom). We'll look at the result we are aiming for first.

Figure 18.3: Simple three-column layout

The idea, in this example, is that you style and label the three screen sections so that they stand out. In other words, these components wouldn't necessarily have any styling in a real ap-

plication since they're used to arrange other components on the screen.

Now, let's take a look at the components used to create this screen layout:

```
import React from "react";
import { Text, View } from "react-native";
import styles from "./styles";
export default function App() {
  return (
    <View style={styles.container}>
      <View style={styles.box}>
        <Text style={styles.boxText}>#1</Text>
      </View>
      <View style={styles.box}>
        <Text style={styles.boxText}>#2</Text>
      </View>
      <View style={styles.box}>
        <Text style={styles.boxText}>#3</Text>
      </View>
    </View>
  );
}
```

The container view (the outermost `<View>` component) is the column and the child views are the rows. The `<Text>` component is used to label each row. In terms of HTML elements,

`<View>` is similar to a `<div>` element, while `<Text>` is similar to a `<p>` element.

Maybe this example could have been called a three-row layout since it has three rows. But, at the same time, the three layout sections are flexing in the direction of the column that they're in. Use the naming convention that makes the most conceptual sense to you.

Now, let's take a look at the styles used to create this layout:

```
import { Platform, StyleSheet, StatusBar } from "react-native";
export default StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    alignItems: "center",
    justifyContent: "space-around",
    backgroundColor: "ghostwhite",
    ...Platform.select({
      ios: { paddingTop: 20 },
      android: { paddingTop: StatusBar.currentHeight }
    })
  },
  box: {
    width: 300,
    height: 100,
```

```
      justifyContent: "center",
      alignItems: "center",
      backgroundColor: "lightgray",
      borderWidth: 1,
      borderStyle: "dashed",
      borderColor: "darkslategray"
    },
    boxText: {
      color: "darkslategray",
      fontWeight: "bold"
    }
  });
```

The `flex` and `flexDirection` properties of `container` enable the layout of the rows to flow from top to bottom. The `alignItems` and `justifyContent` properties align the child elements to the center of the container and add space around them, respectively.

Let's see how this layout looks when you rotate the device from a portrait orientation to a landscape orientation:

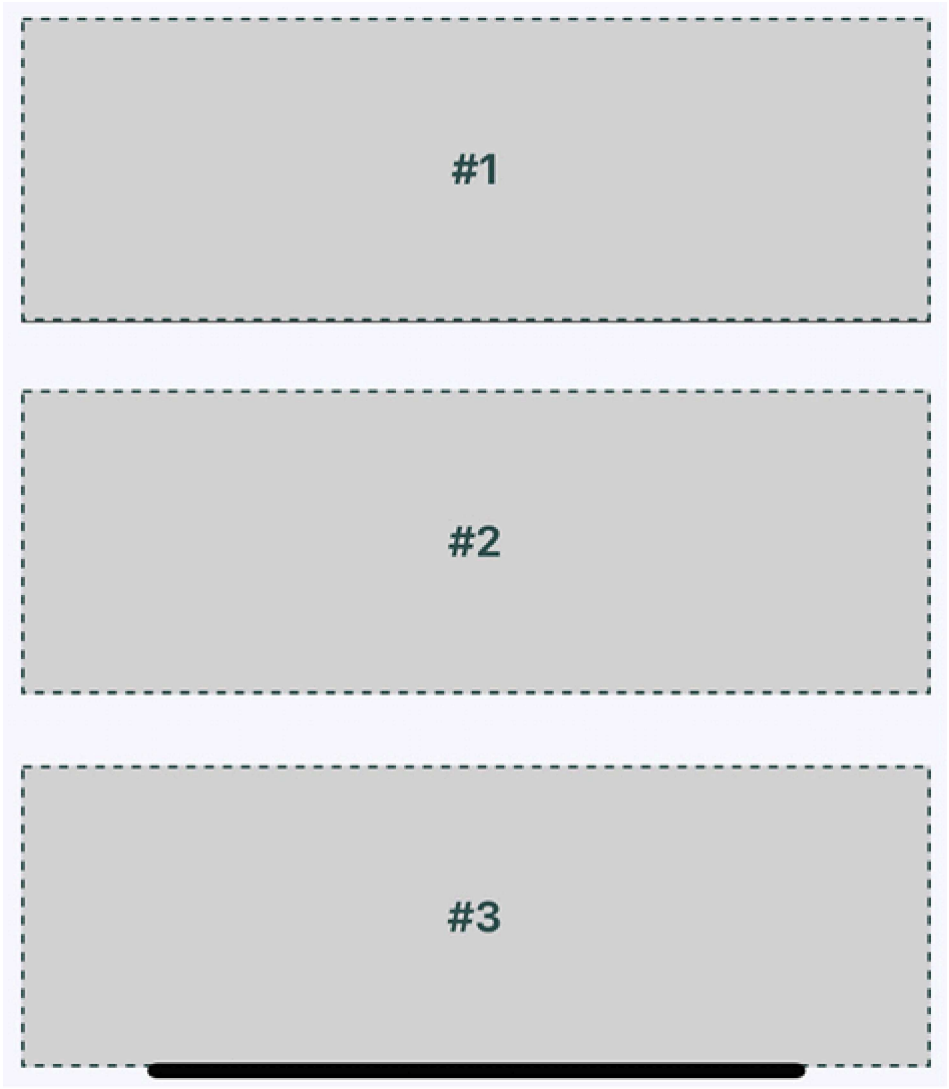

Figure 18.4: Landscape orientation

Flexbox automatically figured out how to preserve the layout for you. However, you can improve on this a little bit. For example, the landscape orientation now has a lot of wasted space

to the left and right. You could create your own abstraction for the boxes that you're rendering. In the following section, we'll improve on this layout.

Improved three-column layout

There are a few things that I think you can improve on from the last example. Let's fix the styles so that the children of the Flexbox could stretch to take advantage of the available space. Do you remember, in the last example, when you rotated the device from a portrait orientation to a landscape orientation? There was a lot of wasted space. It would be nice to have the components automatically adjust themselves. Here's what the new style module looks like:

```
import { Platform, StyleSheet, StatusBar } from "react-native";
export default StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    backgroundColor: "ghostwhite",
    justifyContent: "space-around",
    ...Platform.select({
      ios: { paddingTop: 20 },
      android: { paddingTop: StatusBar.currentHeight },
    }),
  },
  box: {
```

```
      height: 100,
      justifyContent: "center",
      alignSelf: "stretch",
      alignItems: "center",
      backgroundColor: "lightgray",
      borderWidth: 1,
      borderStyle: "dashed",
      borderColor: "darkslategray",
    },
    boxText: {
      color: "darkslategray",
      fontWeight: "bold",
    },
  });
```

The key change here is the `alignSelf` property. This tells elements with the `box` style to change their `width` or `height` (depending on the `flexDirection` of their container) to fill space. Also, the `box` style no longer defines a `width` property because this will be computed on the fly now.

Here's what the sections look like in portrait mode:

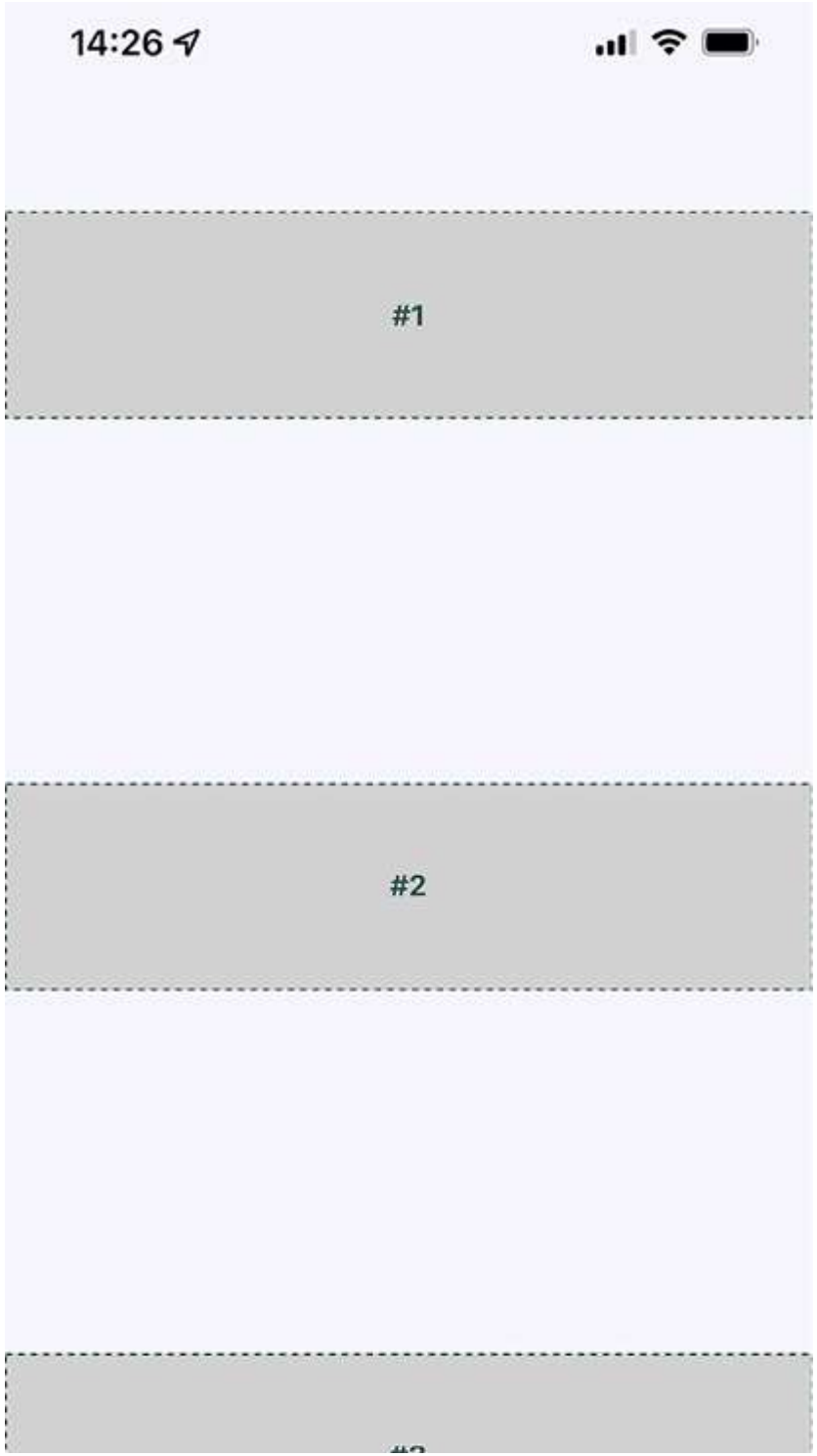
14:26
#1
#2

Figure 18.5: Improved three-column layout in portrait orientation

Now, each section takes the full width of the screen, which is exactly what you want to happen. The issue of wasted space was actually more prevalent in landscape orientation, so let's rotate the device and see what happens to these sections now.

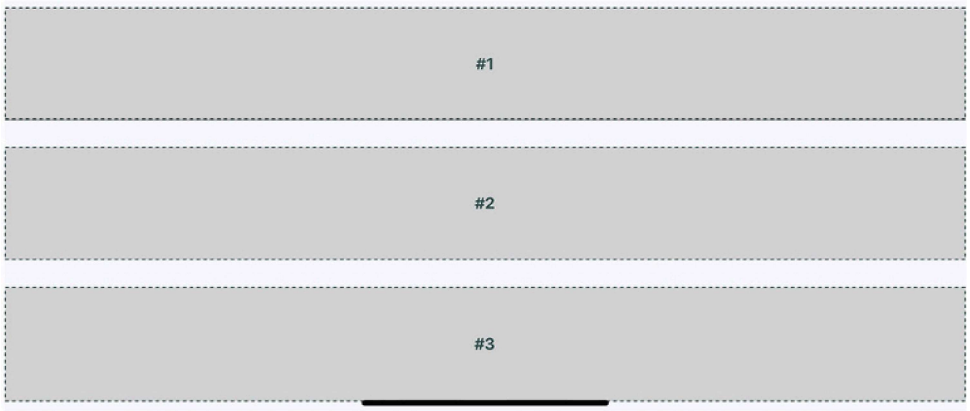

Figure 18.6: Improved three-column layout in landscape orientation

Now your layout is utilizing the entire width of the screen, regardless of orientation. Lastly, let's implement a proper `Box` component that can be used by `App.js` instead of having repetitive style properties in place. Here's what the `Box` component looks like:

```
import React from "react";
import { PropTypes } from "prop-types";
import { View, Text } from "react-native";
import styles from "./styles";
export default function Box({ children }) {
  return (
    <View style={styles.box}>
      <Text style={styles.boxText}>{children}</Text>
    </View>
  );
}
Box.propTypes = {
  children: PropTypes.node.isRequired,
};
```

You now have the beginnings of a nice layout. Next, you'll learn about flexing in the other direction: left to right.

Flexible rows

In this section, you'll learn how to make screen layout sections stretch from top to bottom. To do this, you need a **flexible row**.

Here is what the styles for this screen look like:

```
import { Platform, StyleSheet, StatusBar } from "react-native";
export default StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "row",
    backgroundColor: "ghostwhite",
    alignItems: "center",
    justifyContent: "space-around",
    ...Platform.select({
      ios: { paddingTop: 20 },
      android: { paddingTop: StatusBar.currentHeight },
    }),
  },
  box: {
    width: 100,
    justifyContent: "center",
    alignSelf: "stretch",
    alignItems: "center",
    backgroundColor: "lightgray",
    borderWidth: 1,
    borderStyle: "dashed",
    borderColor: "darkslategray",
  },
  boxText: {
    color: "darkslategray",
    fontWeight: "bold",
```

```
    },
  });
```

Here's the `App` component, using the same `Box` component that you implemented in the previous section:

```
import React from "react";
import { Text, View, StatusBar } from "react-native";
import styles from "./styles";
import Box from "./Box";
export default function App() {
  return (
    <View style={styles.container}>
      <Box>#1</Box>
      <Box>#2</Box>
    </View>
  );
}
```

Here's what the resulting screen looks like in portrait mode:

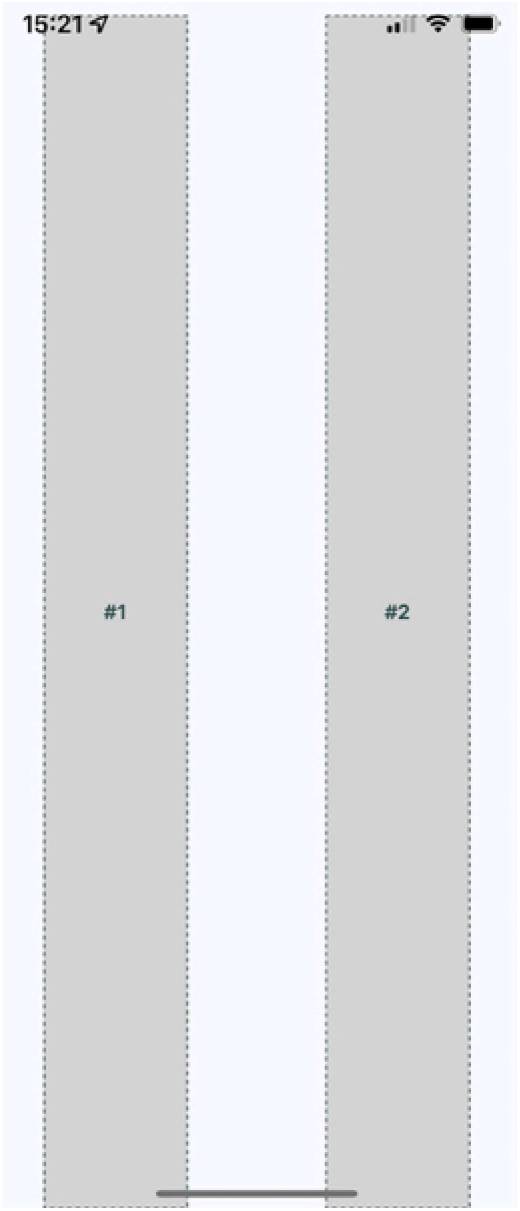

Figure 18.7: Flexible rows in portrait orientation

The two columns stretch all the way from the top of the screen to the bottom because of the `alignSelf` property, which doesn't actually specify which direction to stretch in. The two `Box` components stretch from top to bottom because they're displayed in a **flex row**. Note how the spacing between these two sections goes from left to right? This is because of the container's `flexDirection` property, which has a value of `row`.

Now, let's see how this flex direction impacts the layout when the screen is rotated to a landscape orientation.

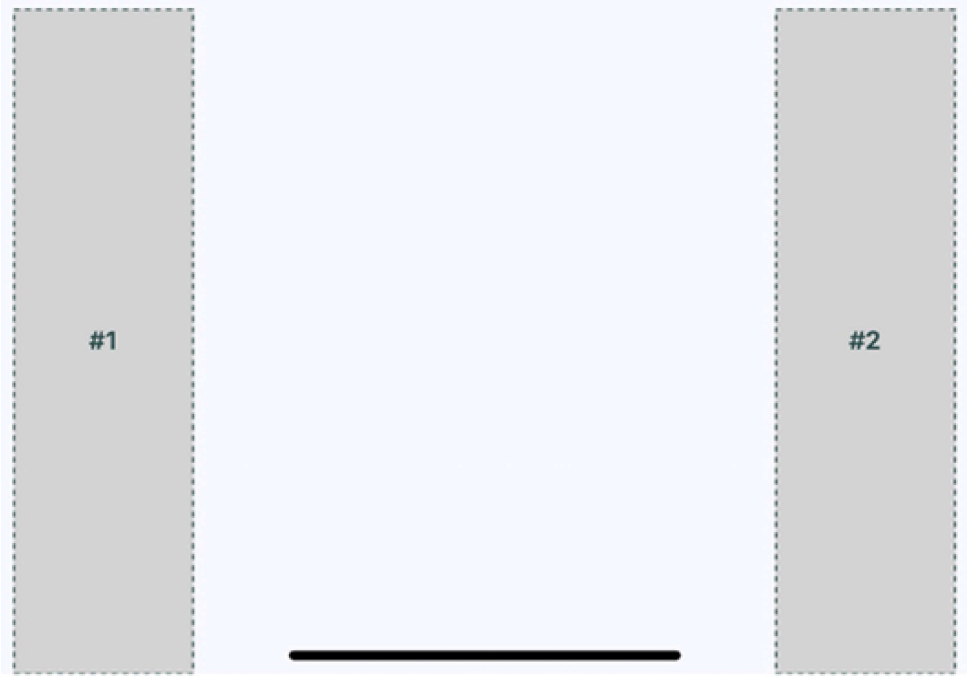

Figure 18.8: Flexible rows in landscape orientation

Since **Flexbox** has a `justifyContent` style property value of `space-around`, space is added proportionally to the left, the right, and in between the sections. In the following section, you'll learn about flexible grids.

Flexible grids

Sometimes, you need a screen layout that flows like a grid. For example, what if you have several sections that are the same width and height, but you're not sure how many of these sections will be rendered? Flexbox makes it easy to build a row that flows from left to right until the end of the screen is reached. Then, it automatically continues rendering elements from left to right on the next row.

Here's an example layout in portrait mode:

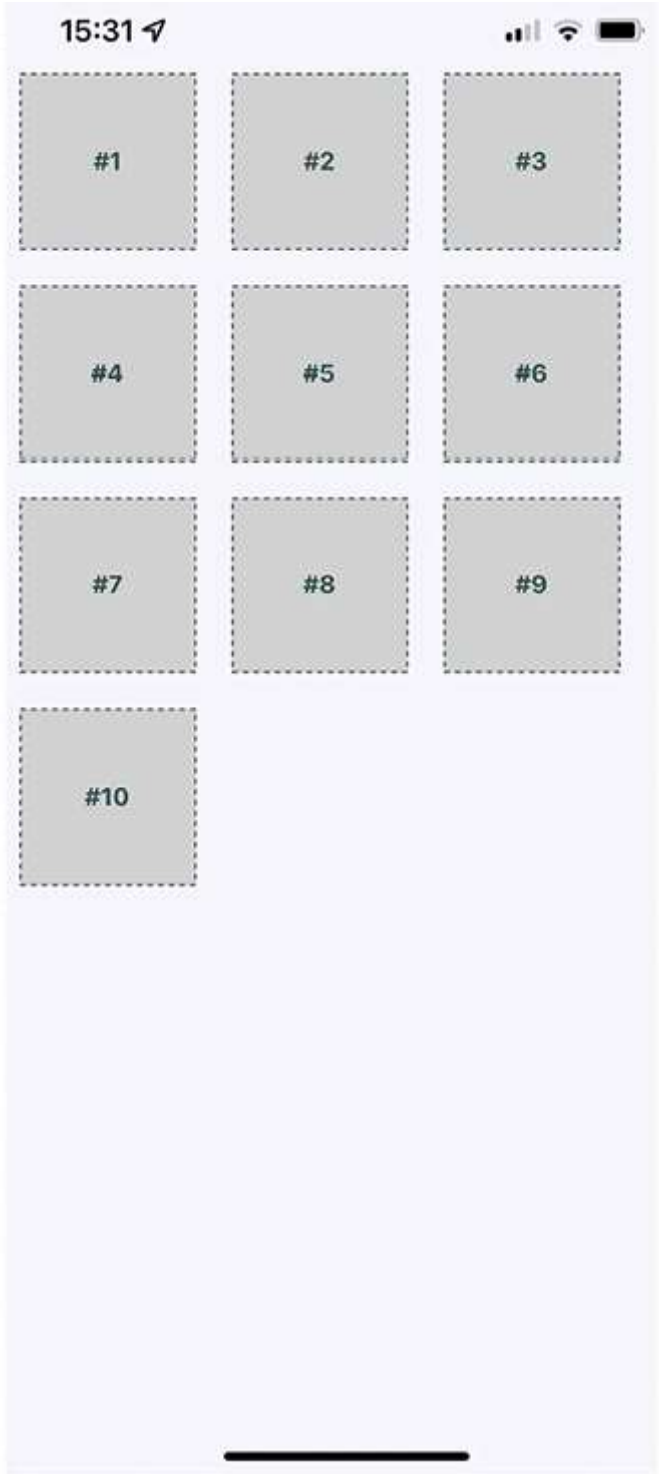
15:31
#1
#2
#3
#4
#5
#6
#7
#8
#9
#10

Figure 18.9: Flexible grids in portrait orientation

The beauty of this approach is that you don't need to know in advance how many columns are in a given row. The dimensions of each child determine what will fit in a given row.

To see the styles used to create this layout, you can follow this link: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter18/flexible-grids/styles.ts**.

Here's the `App` component that renders each section:

```
import React from "react";
import { View, StatusBar } from "react-native";
import styles from "./styles";
import Box from "./Box";
const boxes = new Array(10).fill(null).map((v, i) => i + 1);
export default function App() {
  return (
    <View style={styles.container}>
      <StatusBar hidden={false} />
      {boxes.map((i) => (
        <Box key={i}>#{i}</Box>
      ))}
    </View>
```

```
    );
  }
```

Lastly, let's make sure that the landscape orientation works with this layout:

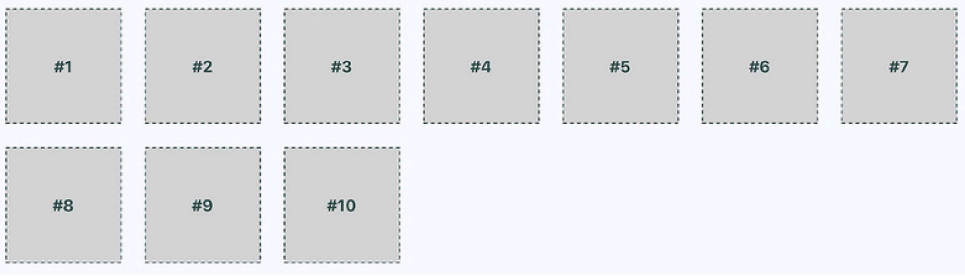

Figure 17.10: Flexible grids in landscape orientation

You may have noticed that there's some superfluous space on the right side. Remember, these sections are only visible in this book because we want them to be visible. In a real app, they're just grouping other React Native components. However, if the space to the right of the screen becomes an issue, play around with the margin and the width of the child components.

Now that you have an understanding of how **flexible grids** work, we'll look at flexible rows and columns next.

Flexible rows and columns

Let's learn how to combine rows and columns to create a sophisticated layout for your app. For example, sometimes, you need the ability to nest columns within rows or rows within columns. To see the `App` component of an application that nests columns within rows, you can follow this link: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter18/flexible-rows-and-columns/App.tsx**.

You've created abstractions for the layout pieces (`<Row>` and `<Column>`) and the content piece (`<Box>`). Let's see what this screen looks like:

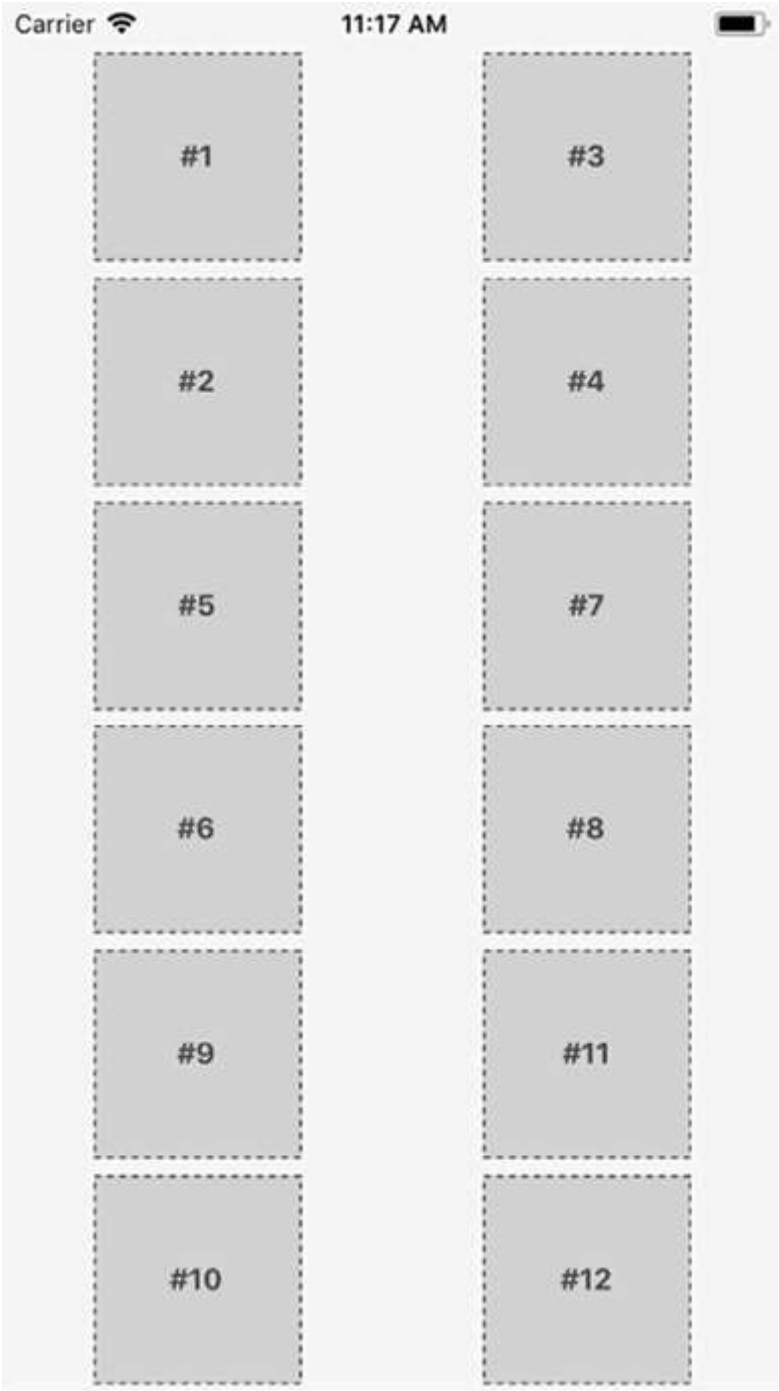
Carrier
11:17 AM
#1
#2
#3
#4
#5
#6
#7
#8
#9
#10
#11
#12

Figure 18.11: Flexible rows and columns

This layout probably looks familiar because you've done it in the *Flexible grids* section. The key difference as compared to *Figure 18.9* is in how these content sections are ordered.

For example, **#2** doesn't go to the right of **#1**, it goes below it. This is because we've placed **#1** and **#2** in `<Column>`. The same happens with **#3** and **#4**. These two columns are placed in a row. Then, the next row begins, and so on.

This is just one of many possible layouts that you can achieve by nesting row Flexboxes and column Flexboxes. Let's take a look at the `Row` component now:

```
import React from "react";
import PropTypes from "prop-types";
import { View, Text } from "react-native";
import styles from "./styles";
export default function Box({ children }) {
  return (
    <View style={styles.box}>
      <Text style={styles.boxText}>{children}</Text>
    </View>
  );
}
Box.propTypes = {
```

```
  children: PropTypes.node.isRequired,
};
```

This component applies the row style to the `<View>` component. The end result is cleaner JSX markup in the `App` component when creating a complex layout. Finally, let's look at the `Column` component:

```
import React from "react";
import PropTypes from "prop-types";
import { View } from "react-native";
import styles from "./styles";
export default function Column({ children }) {
  return <View style={styles.column}>{children}</View>;
}
Column.propTypes = {
  children: PropTypes.node.isRequired,
};
```

This looks just like the `Row` component, just with a different style applied to it. It also serves the same purpose as `Row`: to enable simpler JSX markup for layouts in other components.

Summary

This chapter introduced you to styles in React Native. Though you can use many of the same CSS style properties that you're

used to, the CSS style sheets used in web applications look very different. Namely, they're composed of plain JavaScript objects.

Then, you learned how to work with the main React Native layout mechanism: **Flexbox**. This is the preferred way of laying out most web applications these days, so it makes sense to be able to reuse this approach in a Native app. You created several different layouts, and you saw how they looked in portrait and landscape orientation.

In the next chapter, you'll start implementing navigation for your app.